

Università di Ferrara
Laurea Triennale in Informatica
A.A. 2025-2026
Sistemi Operativi e Laboratorio

Lab-12. I Thread in Java
Prove di esame

Prof. Carlo Giannelli

`https://unife.coursecatalogue.cineca.it/insegnamenti/
2025/46693/2016/9999/10431`

`http://docente.unife.it/carlo.giannelli`

`https://ds.unife.it/team/carlo-giannelli`

Esercizio 1: 1/2

Si realizzi un programma Java multithread che simuli il comportamento di due macchine industriali che con un'interazione ordinata realizzano un prodotto finito in due fasi. A tal fine si implementino le due macchine come un thread «MacchinaA» e un thread «MacchinaB».

Il primo thread «**MacchinaA**» si occupa della prima fase della lavorazione, trasformando (in un ciclo infinito) le materie prime in un primo semilavorato. Il secondo thread «**MacchinaB**» invece lavora sui semilavorati (ricevuti da MacchinaA) per produrre prodotti finiti. Ogni fase della lavorazione (semilavorato e prodotto finito) richiede 200 ms.

Si implementi l'interazione tra le due fasi utilizzando i costrutti Piped per la comunicazione fra thread visti a lezione. Per semplicità si ipotizzi che il semilavorato sia una stringa qualunque che MacchinaA deve passare al thread MacchinaB.

Esercizio 1: 2/2

Il **main-thread** deve occuparsi di creare i due thread e mandarli in esecuzione. Una volta mandati in esecuzione si deve sospendere per 10 secondi e poi terminare l'attività di produzione e attendere l'effettiva terminazione dei thread MacchinaA e MacchinaB.

Si realizzi la comunicazione fra i due thread utilizzando le tre modalità viste a lezione: stream, buffered e object.

Si realizzi la terminazione dei thread MacchinaA e MacchinaB con e senza l'uso del metodo `interrupt()`.

Esercizio 1: traccia

- Definire il thread **MacchinaA** che realizza in un ciclo infinito la produzione di un semilavorato in un tempo di 200 ms. Una volta terminato il semilavorato corrente, MacchinaA scrive una stringa su pipe per comunicare a MacchinaB la produzione del semilavorato. MacchinaA deve esporre il metodo `stopMacchinaA()` per supportarne la sua corretta terminazione.
- Definire il thread **MacchinaB** che realizza in un ciclo infinito la produzione dei lavorati finiti a partire dai semilavorati ricevuti da MacchinaA in un tempo di 200 ms. MacchinaB riceve i semilavorati da MacchinaA attraverso la pipe. MacchinaB deve esporre il metodo `stopMacchinaB()` per supportarne la sua corretta terminazione.
- Definire un metodo **main** all'interno del quale:
 - creare la pipe;
 - creare/attivare i due thread MacchinaA e MacchinaB;
 - attendere 10 s e poi terminare la produzione (ricordandosi di attendere l'effettiva terminazione dei thread).

Esercizio 2: 1/2

Sulla base degli esercizi precedenti si realizzi la seguente applicazione Java thread-safe. In questa versione più sofisticata però i thread **MacchinaA** e **MacchinaB** comunicano attraverso un oggetto condiviso e thread-safe chiamato **ControlloProduzione**.

Il thread **MacchinaA** realizza un semilavorato in un intervallo di tempo variabile tra 400 ms e 500 ms. Una volta realizzato tale semilavorato, **MacchinaA** inserisce tale informazione all'interno di **ControlloProduzione** andando ad incrementare il numero dei semilavorati attualmente disponibili.

Il thread **MacchinaB** trasforma un semilavorato in un prodotto finito. A tal fine **MacchinaB** controlla che all'interno di **ControlloProduzione** sia disponibile almeno un semilavorato. Se tale condizione non è verificata **MacchinaB** si sospende (tramite sleep) per 2 s. Una volta verificata la presenza di un semilavorato, **MacchinaB** decrementa il valore dei semilavorati in **ControlloProduzione** e procede con la creazione di un prodotto finito in un intervallo di tempo tra 100 ms e 150 ms.

Esercizio 2: 2/2

Terminata la produzione, MacchinaB comunica l'evento al **thread-main** utilizzando una pipe. Anche in questo caso si supponga che tale evento venga comunicato utilizzando una stringa arbitraria. Inoltre MacchinaB aggiorna la quantità di lavorati finiti disponibili in ControlloProduzione.

Il thread-main deve creare l'oggetto condiviso ControllaProduzione, 2 thread MacchinaA e 1 thread MacchinaB.

Infine il thread-main dopo essere stato notificato della avvenuta produzione di 15 prodotti finiti termina i thread MacchinaA e MacchinaB e stampa a video il residuo dei semilavorati presenti.

Si verifichi che tali risultati siano corretti.

Esercizio 2: traccia (1/2)

- Definire una classe **ControllaProduzione** che metta a disposizione i metodi per incrementare e decrementare il numero di semilavorati e prodotti finiti in produzione.
- Definire il thread **MacchinaA** che realizza in un ciclo infinito la produzione di un semilavorato in un tempo di 400-500 ms. Una volta terminato il semilavorato corrente, MacchinaA aggiorna il valore dei semilavorati in ControllaProduzione. MacchinaA deve esporre il metodo stopMacchinaA() per consentirne la corretta terminazione.
- Definire il thread **MacchinaB** che realizza in un ciclo infinito la produzione dei lavorati finiti a partire dai semilavorati ricevuti da MacchinaA in un tempo di 100-150 ms. MacchinaB controlla la presenza in ControllaProduzione di almeno un semilavorato. Se tale condizione è verificata procede alla produzione, altrimenti si sospende (con una sleep) per 2 s.

Esercizio 2: traccia (2/2)

- Terminata la produzione, MacchinaB aggiorna il valore dei prodotti finiti in ControllaProduzione e poi **comunica tramite pipe tale evento al thread-main**. MacchinaB deve esporre il metodo stopMacchinaB() per consentirne la sua corretta terminazione.
- Il **thread-main** deve creare una istanza di ControllaProduzione. Poi deve creare due thread di tipo MacchinaA e un thread di tipo MacchinaB. Il thread-main dopo aver ricevuto la notifica dell'avvenuta produzione di 15 prodotti finiti richiede la terminazione dei thread creati (ricordandosi di attendere l'effettiva terminazione dei thread) e stampa a video il numero di semilavorati e prodotti finiti presenti in ControllaProduzione.

Esercizio 3: 1/2

- Si realizzi un programma Java multi-thread per monitorare una linea di produzione. Il thread Main crea una struttura dati condivisa **Macchina** che espone i seguenti metodi: **setMacchinaID()** e **getMacchinaID()** per impostare e ottenere l'ID (int) della macchina da monitorare; **setPezziProdotti()** e **getPezziProdotti()** per impostare e ottenere il numero di pezzi prodotti (int) da una certa macchina.
- **Main** chiede all'utente (via terminale) l'ID della macchina da monitorare e l'obiettivo di produzione (int). Poi, **Main** imposta l'ID della macchina da monitorare e 0 come numero di pezzi prodotti da quella macchina in **Macchina**. Infine, **Main** crea il thread **SimulaProduzione**.

Esercizio 3: 2/2

- ***SimulaProduzione*** aggiorna il numero di pezzi prodotti dalla macchina specificata dall'utente ogni 2 secondi tramite la struttura dati condivisa ***Macchina***. In particolare, ***SimulaProduzione*** aumenta il numero di pezzi prodotti da una certa macchina di un valore intero random compreso tra 0 e 10.

SimulaProduzione deve estendere la classe Thread.

- ***Main*** stampa a video il numero di pezzi prodotti dalla macchina specificata dall'utente ogni 10 secondi. Se l'obiettivo di produzione è stato eguagliato o superato, ***Main*** stampa a video "Obiettivo raggiunto". Dopo 10 rilevazioni, ***Main*** termina ***SimulaProduzione*** con l'opportuno metodo *termina()*. ***SimulaProduzione*** deve terminare immediatamente.

La soluzione deve essere thread-safe.

Esercizio 3: traccia

- Definire una classe **Macchina** che metta a disposizione i metodi per impostare e ottenere l'ID (int) della macchina da monitorare e per impostare e ottenere il numero di pezzi prodotti (int).
- Definire il thread **SimulaProduzione** estendendo la classe Thread. Esso genera un numero intero random compreso tra 0 e 10 ogni 2 secondi attraverso il metodo definito all'interno della classe **Macchina**. In **SimulaProduzione** definire infine il metodo per poter terminare qualora venga ritenuto opportuno.
- In **Main** permettere all'utente di inserire da terminale ID della macchina e obiettivo di produzione (es. utilizzare `BufferedReader`) per poi impostare l'ID della macchina da monitorare e 0 come numero di pezzi prodotti da essa. Definire un ciclo per un totale di 10 iterazioni dove in ognuna si attende 10 secondi prima di stampare a video il numero di pezzi prodotti dalla macchina ed eventualmente uscire dal ciclo se viene raggiunto o superato l'obiettivo di produzione. All'uscita del ciclo terminare il thread **SimulaProduzione**.